Feasibility and Architecture Report: Ultra-Low Latency Conversational AI Voice Agent for Nepalese Restaurants

Executive Overview

The landscape of the global hospitality sector is currently undergoing a profound technological transformation, driven by shifts in consumer behavior, the pervasive integration of the gig economy, and a rising imperative for operational efficiency. Within emerging markets such as Nepal, this transformation is characterized by unique infrastructural challenges, shifting labor dynamics, and a rapidly evolving digital consumer base. The proposition to conceptualize, architect, and deploy an ultra-low latency, conversational Artificial Intelligence (AI) voice agent tailored specifically for the restaurant industry in Nepal represents a highly sophisticated intersection of localized natural language processing, edge computing, and telecommunications engineering. Designed to operate as a fully automated, human-like voice assistant, this initiative specifically targets the automation of phone-based order intake, the processing of menu and price inquiries, the facilitation of complex order customization, and the execution of precise location tracking for delivery logistics.

The primary differentiator of this proposed system is its extreme low latency, which demands an architectural departure from traditional, cloud-reliant AI models that suffer from severe network-induced delays. By leveraging an entirely local, zero-cost technology stack deployed on edge hardware—such as a standard commercial laptop for initial client demonstrations—the operational blueprint neutralizes cloud computing expenditures while practically eliminating internet latency. This report provides an exhaustive, multi-faceted analysis of the total addressable market in Nepal, evaluates the macroeconomic parameters dictating commercial success versus failure, and presents a comprehensive technical schematic for localized GSM telephony routing, Nepali-English code-switching, voice humanization, and administrative management without payment integrations.

Macroeconomic Context and Market Sizing

Understanding the macroeconomic environment and the structural realities of the local commercial ecosystem is fundamental to positioning a technological product of this nature. The Nepalese hospitality industry has expanded significantly over the past decade, evolving from

traditional dine-in models to highly dynamic, delivery-centric operations that rely heavily on telephonic and digital communication.

Total Addressable Market (TAM)

An analysis of the macroeconomic indicators for the Nepalese hospitality sector reveals substantial market depth and a significant capacity for technological absorption. According to the comprehensive National Hotel and Restaurants Survey conducted by the National Statistics Office, the broader hospitality industry—encompassing food and accommodation—generates an excess of Rs326 billion annually, contributing approximately 1.96 percent to the national Gross Domestic Product (GDP). Across the nation, there are exactly 142,223 registered establishments, which include hotels, lodges, resorts, homestays, guest houses, hostels, restaurants, and party venues. This massive figure represents the absolute Total Addressable Market (TAM) for any foundational automation technology targeted at the hospitality sector in Nepal.

The scale of this market is further illuminated by its physical capacity and employment metrics. The survey indicates that restaurants across the country possess a combined single-sitting capacity of 2.23 million seats. Furthermore, the gross fixed assets in Nepal's hospitality establishments are valued at an estimated Rs543.25 billion, with the sector directly employing 106,459 individuals and engaging a total workforce of 387,747 people. The sheer volume of these establishments dictates a high volume of daily transactional interactions, a significant percentage of which are routine inquiries regarding menus, pricing, and takeaway orders.

Serviceable Available Market (SAM)

While the national data provides a macro perspective, a highly specialized conversational AI voice agent designed for delivery and takeaway orders is most applicable to dense urban centers with high smartphone penetration, established delivery infrastructures, and a populace accustomed to ordering food remotely. The Serviceable Available Market (SAM) is therefore heavily concentrated in the Bagmati Province, specifically within the Kathmandu Valley (comprising Kathmandu, Lalitpur, and Bhaktapur).

The National Statistics Office survey highlights that the Bagmati province alone accounts for 52,123 of the national establishments and generates Rs152.076 billion in output, which is more than triple the output of the next largest province. Furthermore, almost half of the nation's restaurant seating capacity—1.03 million seats—is located in Bagmati. Drilling down into the specific urban core, the Revenue and Tax Department of the Kathmandu Metropolitan City (KMC) estimates that approximately 10,000 restaurants and cafés are operating strictly inside the valley. The Restaurant and Bar Association of Nepal (REBAN) reports that there are more than 2,000 quality restaurants, bars, fast food joints, and cafés operating throughout the country,

with over 800 premium establishments situated in Kathmandu alone. These figures delineate a highly concentrated, lucrative SAM where restaurants actively compete for delivery market share and are therefore most likely to invest in operational efficiencies.

Serviceable Obtainable Market (SOM)

The Serviceable Obtainable Market (SOM), representing the immediate, realistic target for early adoption and deployment, consists of mid-to-high-tier restaurants, cloud kitchens, and fast-food franchises in Kathmandu that rely heavily on phone orders but lack the financial resources to maintain dedicated, 24/7 call center staff. According to REBAN, fine dining and tourist-standard restaurants are growing at a rate of 10 percent annually, with about 2,500 such establishments clustered in Kathmandu, Pokhara, and Chitwan.
Capturing even a conservative 5% to 10% of the Kathmandu valley market (approximately 500 to 1,000 restaurants) with a high-margin, low-overhead software-as-a-service or hardware-as-a-service model would constitute a highly lucrative and defensible enterprise. These early adopters are typically characterized by a high volume of inbound calls during peak lunch and dinner hours, resulting in missed calls, abandoned orders, and directly quantifiable lost revenue.

Market Classification	Geographic & Sector Scope	Estimated Volume	Strategic Relevance
Total Addressable Market (TAM)	All hospitality and food establishments across Nepal.	142,223 establishments	Represents absolute theoretical ceiling and total industry volume.
Serviceable Available Market (SAM)	Urban restaurants, cafes, and lodges within the Bagmati Province and KMC.	10,000 to 52,123 establishments	Highly concentrated zone with requisite delivery infrastructure and digital readiness.
Serviceable Obtainable Market (SOM)	Premium quality, fine dining, and heavy-delivery restaurants in	800 to 2,500 establishments	Immediate target demographic experiencing acute labor and volume

Market Classification	Geographic & Sector Scope	Estimated Volume	Strategic Relevance
	Kathmandu.		pain points.

The Competitive Landscape and the "Aggregator Tax"

To successfully penetrate the SOM, the AI voice agent must be positioned against the existing food ordering paradigms in Nepal. The current food delivery ecosystem is heavily dominated by third-party mobile aggregator platforms. Understanding their operational models, their market penetration, and ultimately their friction points with both consumers and restaurant owners is vital for creating a compelling value proposition for direct-to-restaurant AI ordering.

Foodmandu, established in 2010, is widely recognized as the pioneer of the food delivery industry in Nepal. It fundamentally reshaped the socio-economic landscape of urban dining, familiarizing a large segment of the population with online transactions and app-based services. Foodmandu delivers food from over 800 restaurants across three cities in the Kathmandu Valley, utilizing a pool of over 200 riders. Following Foodmandu's trailblazing path, the ride-sharing giant Pathao launched Pathao Food, extending its vast logistics network to connect with more than 1,000 restaurants. Other significant players include Bhoj Deals (formerly Bhojdeals), which expanded its services to over 800 restaurants across Kathmandu, Bhaktapur, and Lalitpur, and Khaanpin, which differentiated itself by offering a 24-hour delivery system catering to late-night consumers.

While these platforms provide restaurants with a significantly expanded customer reach and handle the complexities of last-mile logistics, they extract an enormous amount of value from the transaction, creating a structural friction known in the industry as the "aggregator tax." These platforms typically charge restaurants steep commission rates on every single order, which can range anywhere from 15% to 25% of the total ticket value. Simultaneously, they charge the end consumer delivery fees; for example, Pathao has been noted to charge flat delivery fees of Rs 150, while others calculate fees based on distance and order minimums.

Furthermore, consumer sentiment regarding these applications indicates significant room for disruption. Despite their market dominance, the applications suffer from structural and technical inadequacies. Foodmandu's mobile application currently holds a 2.8 rating on the application store, with extensive consumer reviews citing severe late delivery times, application bugs, app freezing, system failures to update orders, and persistent issues with delivery riders failing to locate pinned addresses on maps. Similarly, Bhoj Deals holds a 2.9 rating, with customers reporting that the application crashes during payment integrations with digital wallets like eSewa, and frequently fails to deliver orders on time.

These intersecting factors—exorbitant restaurant commissions, high consumer delivery fees, and buggy digital interfaces—create the perfect commercial wedge for an ultra-low latency AI voice agent. A significant portion of the consumer base still prefers the immediacy and perceived reliability of simply calling a restaurant directly. By deploying a conversational AI that answers these phone calls instantly, restaurants can process direct orders automatically, entirely bypassing the aggregator platforms and immediately reclaiming their 15% to 25% profit margins.

Labor Economics and Operational Pain Points

Beyond the financial drain of aggregator commissions, the Nepalese hospitality sector is currently facing an acute and escalating crisis regarding human capital. The Hotel Association of Nepal has publicly stated that the growth of the nation's tourism and hospitality sector is being actively threatened by a severe labor shortage. This shortage is primarily driven by mass foreign employment migration, where young, capable Nepalese workers are leaving the country for more lucrative opportunities in the Middle East, Southeast Asia, and Eastern Europe.

This macro-economic labor drain has micro-economic consequences for every restaurant in Kathmandu. Finding, training, and retaining reliable front-of-house staff to manage phone lines, answer customer inquiries, and input orders into a Point of Sale (POS) system has become increasingly difficult and expensive. High staff turnover means that restaurant owners are perpetually engaged in a cycle of retraining. When a human receptionist is overwhelmed during the peak dining hours of 7:00 PM to 9:00 PM, phone lines ring out, callers are put on extended hold, and potential high-margin direct orders are lost to competitors.

The conversational AI voice agent directly addresses this systemic labor crisis. Unlike human staff, the AI does not require training on new menu items (as it instantly reads from a database), it does not take breaks, it does not suffer from fatigue during peak hours, and it never resigns. It can handle concurrent calls (if the telecom infrastructure permits) and operates with absolute consistency. By framing the product not merely as an IT upgrade, but as an immunized labor asset that solves the most pressing operational headache for restaurant owners, the sales narrative becomes extraordinarily compelling.

Evaluating Success and Failure Paradigms

Deploying an advanced conversational AI system in an emerging market characterized by infrastructural volatility carries distinct risks and immense potential rewards. Determining whether this specific initiative will fail or succeed requires a highly nuanced evaluation of local constraints, specifically regarding telecommunications quality, linguistic diversity, and user psychology.

Pathways to Failure

The primary risk of failure for this endeavor stems from acoustic and linguistic degradation. Nepalese consumers frequently communicate in environments with high levels of background noise, such as heavy urban traffic, crowded public squares, or multi-generational households. This ambient noise severely degrades the audio signal quality before it even reaches the AI's transcription engine. Furthermore, telecom networks in Nepal, operated primarily by Nepal Telecom (NTC) and Ncell, can suffer from packet loss, jitter, and low-fidelity audio compression during peak network congestion. If the localized speech-to-text engine cannot reliably transcribe distorted, low-bandwidth, 8kHz audio, the AI will fail to understand the customer, leading to a loop of repeated questions, user frustration, and immediate call abandonment.

A secondary pathway to failure involves the mishandling of code-switching. If the AI forces the caller to speak in pure, formal English, or pure, formal Nepali, it will alienate the target demographic. Urban Nepalese consumers speak a highly fluid mixture of both languages. If the Natural Language Understanding (NLU) model fails to parse a sentence like, "Euta mixed pizza ra dui ota cold coffee order garne, address chahi Patan Durbar Square nira," the conversational illusion breaks, and the system fails as a commercial product.

A final pathway to failure is commercial friction. If the product is marketed as a complex, cloud-based SaaS product requiring massive upfront integration fees, restaurant owners—who operate on notoriously thin margins and may possess limited technical literacy—will reject it. If the system requires them to maintain complex routing tables, manage API keys, or deal with international payment gateways for cloud services, adoption will stall entirely.

Pathways to Commercial Success

Conversely, the probability of immense success is anchored in the solution's financial proposition and its frictionless deployment model. The success of this product hinges entirely on its ability to demonstrate immediate, undeniable return on investment (ROI). If the AI agent can successfully process just five to ten direct orders a day for a mid-tier restaurant, the savings generated by bypassing third-party aggregator commissions will instantly justify the monthly cost of the software.

To maximize this success, the go-to-market strategy must leverage the localized demonstration architecture requested. By walking into a restaurant with a standard commercial laptop, plugging in a standard cellular modem (dongle), and allowing the restaurant owner to physically dial the AI and place a test order in real-time, the abstract concept of Artificial Intelligence is rendered immediately tangible and localized. This aggressive, high-touch, "Hardware-as-a-Service" sales motion bypasses technological skepticism.

Furthermore, success will be solidified by ensuring the system requires zero behavioral change from the restaurant's existing customer base. Customers do not need to download a new

application, navigate a buggy interface, or register for an account. They simply dial a standard 10-digit mobile number, just as they have always done, and speak naturally.

Zero-Cost Technical Architecture and Local Deployment

Fulfilling the explicit requirement to design an entirely free, local technology stack that facilitates a demonstration on a standard laptop involves orchestrating several advanced open-source components. This architecture must handle raw telephony hardware, real-time audio streaming, speech-to-text transcription, complex natural language understanding, and text-to-speech synthesis without invoking external paid APIs such as OpenAI, Google Cloud, or Twilio. By keeping the compute entirely local, the system achieves zero marginal cost per call and ensures privacy.

The Core Technology Stack

The technological foundation relies heavily on Python for orchestrating the intensive artificial intelligence workloads, and React.js for providing the user-facing administrative interface. These components are connected via a lightweight, serverless SQLite database. This specific stack avoids all recurring enterprise licensing fees and cloud hosting costs, making it ideal for the capital-constrained startup phase.

System Component	Technology Choice	Functionality & Strategic Rationale
Telephony Gateway	Asterisk PBX + chan_dongle	Open-source PBX software that routes real cellular calls through a physical USB 3G/4G modem directly into the local server environment.
Backend Framework	Python (FastAPI)	Highly performant, asynchronous framework that orchestrates real-time audio streaming, model inference pipelines, and

System Component	Technology Choice	Functionality & Strategic Rationale
		state management.
Speech-to-Text (ASR)	Whisper.cpp (Local/Quantized)	Translates incoming 8kHz narrowband audio to text. Using the C++ port minimizes CPU/GPU load, allowing it to run on standard laptop hardware.
Language Model (LLM)	Qwen2.5 / Llama 3.2 (Local)	Handles complex dialogue logic, dynamic menu parsing, and contextual intent extraction from highly mixed language inputs.
Text-to-Speech (TTS)	Piper TTS / Coqui TTS	Generates human-like voice responses entirely locally with extremely low latency, bypassing network round-trips.
Frontend Dashboard	React.js + Tailwind CSS	Provides a responsive, zero-configuration dashboard for restaurant staff to update menus and view incoming orders visually.
Database	SQLite	Stores menu items, historical prices, and call logs locally as a single file,

System Component	Technology Choice	Functionality & Strategic Rationale
		eliminating the need for complex database server management.

Engineering for Ultra-Low Latency

The defining differentiator of this voice agent, as stipulated, is speed. Human conversational latency—the gap between one person stopping speaking and the other starting—typically hovers around 200 to 300 milliseconds. If an AI takes longer than 1,000 milliseconds to respond, the caller will subconsciously assume the line is dead, repeat themselves, or become frustrated, completely breaking the conversational illusion. Achieving ultra-low latency on a local laptop without the massive compute clusters of cloud providers requires rigorous, uncompromising optimization of the entire data pipeline.

The architecture must abandon the traditional request-response REST API model in favor of continuous, bidirectional WebSockets or raw TCP streaming. As the user speaks into their mobile phone, the audio must be streamed in tiny micro-chunks (e.g., 50 to 100 milliseconds) directly from the telephony gateway into the Automatic Speech Recognition (ASR) engine. Simultaneously, the system must employ a highly aggressive Voice Activity Detection (VAD) algorithm. Silero VAD, a lightweight enterprise-grade model, can instantly detect the exact millisecond a user has stopped speaking, ignoring background traffic noise. Once the VAD signals the end of speech, the final transcription is immediately fed into the local Large Language Model.

Recent advancements in open-weight models, specifically Llama 3.2 and Qwen2.5, offer exceptional multilingual capabilities and can be heavily quantized (compressed) to 4-bit formats to run natively on consumer laptop GPUs or even high-end multi-core CPUs. To further reduce latency, the LLM must be configured to stream its output token-by-token, rather than waiting to generate the entire paragraph.

The Text-to-Speech (TTS) engine is then pipelined into this stream. It must not wait for the entire sentence to be generated. Instead, as soon as the LLM outputs a complete phrase or semantic clause (separated by a comma, period, or conjunction), the TTS engine instantly synthesizes that specific chunk and streams the raw audio back to the telephony gateway. By heavily pipelining these processes—transcription, inference, and synthesis—the time-to-first-byte (TTFB) of audio reaching the caller can be compressed to sub-800 milliseconds, closely mimicking human reflexivity and ensuring a fluid conversational dynamic.

Telephony Routing in Nepal: The GSM Gateway Solution

A critical bottleneck in developing voice AI for emerging markets is the lack of programmable cloud telephony infrastructure. In Western markets, developers effortlessly use platforms like Twilio or Plivo to purchase virtual phone numbers and route calls over the internet via SIP trunks. In Nepal, telecommunications regulations make acquiring programmable SIP trunks highly restricted, expensive, and subject to intense bureaucratic friction. Furthermore, relying on international SIP providers introduces massive network latency, as a local call made in Kathmandu would be routed to servers in Singapore or the United States before returning to the laptop, adding hundreds of milliseconds of delay and destroying the low-latency requirement.
To bypass this infrastructural hurdle and achieve the zero-cost, local demonstration requirement, the architecture must utilize a custom GSM Gateway built using standard, widely available consumer hardware.

The Asterisk and chan_dongle Architecture

The solution involves utilizing a standard 3G/4G USB modem (frequently referred to as a data dongle), such as the Huawei E173, E1752, E1762, or ZTE series devices (e.g., K3565-Z, MF190). These specific devices are inexpensive, readily available in the local electronics market, and accept standard NTC or Ncell physical SIM cards.
When these USB modems are plugged into a Linux-based laptop or a Raspberry Pi, they typically act as a mass-storage device by default, attempting to install Windows drivers. Using a crucial Linux utility called usb_modeswitch, the device is forced to drop its storage partition and expose its internal serial interfaces. By executing specific vendor and product hexadecimal codes (e.g., usb_modeswitch -v 0x12d1 -p 0x140c), the device switches into "modem-only mode". This exposes the diagnostic, GPS, and critical audio serial ports, which typically mount as /dev/ttyUSB1 for the raw audio stream and /dev/ttyUSB2 for data and AT commands.
The core of the telephony routing relies on Asterisk, the industry-standard open-source Private Branch Exchange (PBX) software. To interface Asterisk directly with the USB modem, a specific, highly specialized channel driver called chan_dongle is utilized. While chan_dongle is an unsupported third-party module and not officially maintained by the core Asterisk project, it has been extensively documented, modified, and successfully implemented by the open-source telecommunications community to bridge physical cellular audio directly into digital VoIP environments.
Once compiled and configured (via the /etc/asterisk/dongle.conf file), the entire communication flow is handled locally without ever touching the public internet:

1. A customer dials the standard Ncell or NTC phone number associated with the physical

SIM card inserted into the Huawei USB dongle.
2. The modem receives the GSM cellular signal from the local cell tower.
3. The chan_dongle driver captures the incoming call via the data port and routes the raw, two-way audio stream from the audio port directly into the Asterisk dialplan.
4. Asterisk immediately answers the call (using the Answer() application within extensions.conf) and opens a two-way audio socket to the local Python FastAPI server using tools like MixMonitor or External Media.
5. The Python server ingests the audio chunk by chunk, processes the AI logic, and streams the synthesized audio response back through Asterisk and out via the cellular network.

This architecture is genuinely revolutionary for a localized AI startup. It requires absolutely zero cloud infrastructure, incurs zero monthly API fees for virtual phone numbers (requiring only standard local SIM validity recharges), and guarantees the lowest possible audio latency because the entire transaction—from the radio wave reception to the AI inference—occurs within the physical laptop sitting on the restaurant manager's desk.

Mitigating Hardware and Audio Quirks

Operating raw telephony hardware requires specific mitigations to ensure enterprise-level reliability. USB modems can occasionally drop their network registration due to thermal throttling or minor tower fluctuations. The chan_dongle configuration allows for periodic network checks to ensure the device remains registered and actively sends AT reset commands if the connection drops.

Furthermore, the audio captured from a standard GSM network is sampled at 8,000 Hz (narrowband audio). Modern AI models are often trained on 16kHz or 24kHz wideband audio. The local Whisper ASR model must be specifically loaded with weights optimized for 8kHz, or the incoming audio must be mathematically upsampled. Feeding low-resolution 8kHz audio into a model expecting 16kHz without proper ALSA/SoX formatting will result in severe transcription hallucinations, rendering the system useless.

Overcoming Natural Language Constraints: Nepali-English Code-Switching

One of the most complex engineering challenges for deploying conversational AI in Kathmandu is purely linguistic. The target demographic does not interact in pure, formal Nepali, nor do they speak pure English. They utilize a complex, highly fluid, and contextual code-switching dialect colloquially referred to as "Nepanglish."

A typical customer order might sound like: "Momo ko euta order garne ho, make it very spicy, ani location chahi Thamel Narsingh Chowk ho." The AI must seamlessly parse this mixed

syntax, identifying "Momo" as the core entity, "euta" (one) as the quantity, "spicy" as the modifier, and "Thamel Narsingh Chowk" as the geographical variable.

Multilingual Model Selection and Prompt Engineering

To handle this extreme code-switching without relying on expensive, latency-heavy paid APIs like OpenAI's GPT-4, the system must utilize the absolute cutting edge of open-source models. Both Qwen2.5 and Llama 3.2 have recently demonstrated state-of-the-art proficiency across diverse multilingual benchmarks, outperforming older open models significantly. Qwen2.5, developed with a vast multilingual training corpus, possesses a uniquely robust understanding of transliterated roman text (the practice of writing Nepali words using the standard English alphabet).

The language model must be strictly grounded using highly specific system prompts and constraints. The prompt must explicitly instruct the model to expect romanized Nepali mixed with English culinary terminology. Furthermore, the prompt dictates the precise personality, tone, and operational boundaries of the agent, ensuring it does not hallucinate menu items that do not exist in the restaurant's offerings.

By utilizing Retrieval-Augmented Generation (RAG) techniques, the agent queries the local SQLite database in real-time based on the transcribed text. When a user asks, "Kati parcha chicken momo lai?" (How much is the chicken momo?), the LLM translates the semantic intent, queries the local database for the "Chicken Momo" row, retrieves the integer price, and generates a natural language response: "Chicken momo ko price two hundred and fifty rupees ho. Would you like to add it to your order?" This ensures absolute factual accuracy regarding pricing and availability.

Voice Humanization and Prosody

A robotic, monotone voice will alienate users, especially within a hospitality context that implicitly demands warmth, politeness, and social grace. To humanize the voice agent, the local TTS engine must be carefully tuned to mimic human conversational imperfections.

Humanization involves inserting deliberate pauses, breaths, and conversational fillers into the audio output. The LLM can be instructed via its system prompt to occasionally generate localized conversational fillers—such as "Umm," "Hajur," "Tesaibhae," or "La"—before processing complex requests. For instance, if the database query or the LLM inference takes an extra 300 milliseconds, the system can instantly trigger a pre-rendered, highly natural audio file saying "Ek chin hai..." (Just a moment...) to fill the dead air. This psychological trick masks any underlying computational latency and reinforces the illusion of human interaction.

Furthermore, the acoustic model underlying the TTS engine should ideally be fine-tuned on a dataset of native Nepalese speakers speaking English. Utilizing a default American or British

TTS voice (such as those standard in Piper TTS) to pronounce Nepalese food items ("Choila," "Sekuwa," "Chatamari") or neighborhood names ("Boudha," "Jhamsikhel") will sound jarring, alien, and immediately expose the system as a bot. Fine-tuning ensures the prosody and accent match local expectations.

Location Tracking and Order Fulfillment Mechanics

Once the AI has successfully navigated the menu, executed order customizations (e.g., "no onions," "extra spicy"), and confirmed the final order cart, it must secure the delivery location. This presents a unique and notoriously difficult logistical challenge in Kathmandu, where formalized street addresses, standardized postal codes, and sequential house numbers are largely non-existent.

Contextual Location Parsing

Instead of rigid street addresses, Nepalese residents navigate via a complex web of landmarks, neighborhood names, prominent buildings, and intersections (chowks). The AI must be specifically prompted to handle this geographical reality. If a user states, "Mero ghar Baneshwor ma ho, Eyeplex Mall ko thik pachadi" (My house is in Baneshwor, exactly behind Eyeplex Mall), the LLM must extract "Baneshwor" as the macro-zone and "behind Eyeplex Mall" as the micro-routing instruction.

Because the voice channel is notoriously poor for conveying exact digital coordinates, the system must utilize a multimodal fallback mechanism to guarantee delivery success. Since the entire architecture is physically connected to a cellular SIM card via the GSM dongle, the AI automatically possesses the caller's exact mobile number via the standard Caller ID stream provided by Asterisk upon ring.

Upon confirming the food items, the AI can state: "I have confirmed your order. Because exact locations can be tricky, I am sending you an SMS right now. Please click the link to share your exact map location."

Simultaneously, the Python backend intercepts the conversation state and executes a raw AT command (specifically AT+CMGS="Phone_Number" "Message") through the Huawei modem's data port (/dev/ttyUSB2). This triggers the modem to instantly send a standard SMS to the caller while they are still on the phone. The SMS contains a deep link to a lightweight, free local endpoint (hosted via Ngrok or a simple dynamic DNS service during the laptop demonstration phase). When the user taps the link, a simple webpage asks for GPS permission and drops a pin, sending the exact latitude and longitude back to the Python backend. This elegant solution bridges the gap between the limitations of voice AI and the absolute logistical necessity of precise geographic coordinates, entirely bypassing the need to integrate or pay for expensive

third-party mapping APIs.

The Administrative React Panel (Zero Payment Integration)

To ensure that restaurant owners can maintain the system without requiring constant technical intervention from the developers, the solution requires a robust, locally hosted administrative dashboard. Built using React.js and styled with Tailwind CSS for a modern, responsive feel, this panel is served directly from the local laptop or edge server on a local network (e.g., localhost:3000 or a designated LAN IP address).
This dashboard features three primary modules:

1. **Menu Management:** A straightforward CRUD interface interacting directly with the local SQLite database. Restaurant managers can add new dishes, adjust prices, or mark items as "out of stock" in real-time. These changes instantly update the LLM's RAG context.
2. **Live Call Dashboard:** A WebSocket-powered view showing active calls, real-time speech-to-text transcriptions, and the AI's generated responses. This allows managers to monitor AI performance and step in manually if the system misinterprets an order.
3. **Order Kanban Board:** Incoming orders populate a visual kanban board (New, Cooking, Ready, Out for Delivery). Each card contains the exact items, the user's phone number, and a clickable GPS pin captured via the SMS fallback mechanism.

Crucially, this system enforces a "Zero Payment Integration" policy. By intentionally omitting digital wallet (eSewa, Khalti) or card gateways, the system bypasses complex regulatory compliance, transaction fees, and webhook latency. All orders are strictly "Cash on Delivery" (COD), which aligns perfectly with current consumer habits in Nepal and keeps the technology stack entirely decoupled from financial liability.

What to Make: Bill of Materials & System Architecture

To successfully build and deploy this system for a restaurant demonstration, specific hardware and software components must be assembled. This represents the minimum viable setup required for edge deployment.

Hardware Requirements (The Edge Node)

Component	Specification / Model	Purpose
Demonstration Laptop	Standard PC (Intel i5/i7, 16GB RAM) or Mac (M1/M2)	Acts as the local edge server to process LLM and TTS inferences without cloud latency.
GSM USB Modem	Huawei E173, E1752, or E1762	Interfaces physical SIM card calls into the laptop's digital VoIP environment.
SIM Card	NTC or Ncell 4G SIM	Provides the public 10-digit number for customers to dial.

System Flow Diagram (Data Pathway)

The following representation illustrates the raw data flow from a customer's mouth to the AI and back, ensuring sub-second latency via a zero-cost local stack.

```
[ Customer Phone (NTC/Ncell) ]
        | (GSM Cellular Network)
        v
[ Huawei USB Modem / Dongle ] --> (Raw Audio 8kHz via /dev/ttyUSB1)
        |
        v
[ Asterisk PBX (chan_dongle) ] --> (Converts raw audio to TCP Stream)
        |
        v
[ Python Backend (FastAPI) ]
        |--> 1. Silero VAD (Detects speech end instantly)
        |--> 2. Whisper.cpp (Transcribes 8kHz audio to text locally)
        |--> 3. Llama 3.2 / Qwen2.5 (Generates response & queries SQLite
Menu)
        |--> 4. Piper TTS (Synthesizes text to human-like voice)
```

```
        |
        v
[ Asterisk PBX ] <-- (Receives generated voice stream)
        |
        v
[ Huawei USB Modem ] --> (Broadcasts voice back to customer)
```

How to Start: Implementation Blueprint

Follow these sequential steps to rapidly prototype and deploy the system in a controlled environment:

1. **Procure Hardware and Unlock the Modem:**
    - Purchase a compatible Huawei USB modem from local tech markets in Kathmandu (e.g., New Road, Putalisadak).
    - Insert an active, registered NTC or Ncell SIM card.
    - Connect it to a Linux environment (Ubuntu recommended) and use usb_modeswitch to force the modem into diagnostic/serial mode, exposing /dev/ttyUSB0, /dev/ttyUSB1, and /dev/ttyUSB2.
2. **Configure Asterisk and chan_dongle:**
    - Install Asterisk PBX on the Linux machine.
    - Compile and install the chan_dongle module from source.
    - Configure dongle.conf to recognize the IMEI and audio ports of your specific modem.
    - Set up the Asterisk dialplan (extensions.conf) to automatically answer incoming calls and pipe the two-way audio via WebSocket or raw TCP to a local port.
3. **Initialize the Local AI Pipeline:**
    - Set up a Python virtual environment and install FastAPI.
    - Download a quantized model of Whisper.cpp optimized for edge deployment.
    - Download a 4-bit quantized version of Qwen2.5 or Llama 3.2 via Ollama or llama.cpp. Craft a strict system prompt teaching the LLM the restaurant's menu and instructing it to gracefully handle code-switching ("Nepanglish").
    - Install Piper TTS and download a clear, high-speed voice model.
4. **Build the SQLite Database and React Dashboard:**
    - Create a simple SQLite database schema with tables for Menu_Items, Orders, and Call_Logs.
    - Develop a React.js frontend that fetches data from the FastAPI backend, allowing

non-technical users to add local items like "Chicken Momo" or "Dal Bhat" to the menu.

5. **Test and Iterate (The "Friends and Family" Phase):**
    - Turn on the servers. Have a team member dial the SIM card's phone number.
    - Speak in mixed Nepali-English. Monitor the terminal logs to measure the exact millisecond latency between speech-end and the TTS response.
    - Refine the VAD (Voice Activity Detection) threshold to ensure background noise from Kathmandu's streets doesn't trigger false transcriptions.

By adhering to this localized, hardware-centric methodology, the project neutralizes cloud costs, completely eliminates aggregator commissions for the restaurant, and solves the pressing labor shortages in the Nepalese hospitality sector.